EVOLVING 8-BIT ARRANGEMENTS: A GENETIC ALGORITHM FOR THE NES FOUR-VOICE IDIOM

Giovanni Martinelli
Departamento de Ciência da Computação, Universidade Federal de Minas Gerais

ABSTRACT

An 8-bit cover is a re-composition: the Nintendo Entertainment System (NES) sound chip plays only four monophonic voices, so chords must become arpeggios and a bass line and a drum part must be invented. We present a genetic algorithm (GA) that arranges a lead sheet (melody, chord symbols and part labels) for the four NES channels. The genotype holds four integer genes per bar, which select the harmony texture of the second pulse channel, the bass texture of the triangle channel, the drum pattern of the noise channel and the octave of the melody. Deterministic realizers turn every genotype into a valid arrangement, so the chip rules hold by construction, and a weighted mean of seven music-theory criteria scores each plan. Selection is roulette with windowing and elitism, crossover cuts between bars, and mutation changes 1% of the genes. We arrange two traditional tunes with famous 8-bit versions, *Ievan Polkka* and *Korobeiniki* (the Tetris theme), and a Brazilian song reduced from a multi-track MIDI file, and report the evolution of the fitness, the diversity across seeds, a random-search baseline and a weight ablation.

1. INTRODUCTION

Fans re-create songs as 8-bit music, the sound of 1980s game consoles. The NES audio processing unit offers two pulse channels (P1, P2), a triangle channel (TR) and a noise channel (NO), each playing one note at a time [1]. Within these limits, chiptune composers settled on a few conventions: fast arpeggios that stand for chords, a dedicated bass voice and noise-based percussion [2]. These were aesthetic choices as much as technical necessities [3]. Arranging a song for the chip is therefore a composition task: the melody is given, but the harmony texture, the bass line, the drum part and the register must be decided bar by bar.

We use the NES four-voice idiom as the stylistic constraint of a classic algorithmic composition method, the genetic algorithm. The system receives a lead sheet in ABC notation and outputs a four-track MIDI file that obeys the chip rules, rendered to audio only for listening. To our knowledge, no earlier GA arranges music for the NES chip; the contribution is the idiom as a searchable space of textures, while the GA itself is standard.

© G. Martinelli. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). Course abstract for DCC831 (UFMG, 2026/2), formatted with the ISMIR 2026 template; it was not submitted to ISMIR.

2. RELATED WORK

Genetic algorithms have been used to arrange existing music for an instrument with physical limits, as in the guitar arranger of Tuohy and Potter [4]; our system shares this view of arranging as a search problem, but targets the four monophonic channels of the NES and generates new accompaniment from the chord symbols instead of selecting notes from a full score. Evolutionary orchestration searches Pareto-optimal combinations of instruments under instrumentation constraints [5]; we address a symbolic counterpart of this problem with a single weighted fitness. In melody harmonization, Freitas and Guimarães evolve one chord per bar with a crossover that cuts only at bar boundaries [6], and Phon-Amnuaisuk et al. report that the rule weights, more than the operator rates, decide the quality of evolved four-part harmonies [7]; we adopt the bar-aligned crossover and report a weight ablation. GenJam organizes its genotype in bars and four-bar phrases [8], and AMUSE combines normalized music-theory objectives in a weighted sum [9]; our fitness follows the same scheme, with criteria about the interplay between channels. Evolutionary work on 8-bit music has evolved bytebeat sound formulas with human ratings [10], and LakhNES maps arbitrary MIDI parts onto the NES voices at random as data augmentation [11]; instead, we search the per-bar textures with explicit musical criteria. A GA with roulette selection and elitism also steered a neural language model in [12].

3. METHOD

3.1 Representation

The lead sheet is parsed into 4/4 bars. Each bar holds its melody notes (MIDI pitch, onset and duration in beats), its chord segments (root and chord tones), a part label mapped to a role (verse or chorus) and a phrase index (phrases of four bars). The genotype is an integer array with one row per bar and four genes per row (Table 1). For each gene value, a deterministic *realizer* writes the notes of the bar: arpeggios cycle the chord tones from the root in the register C3–B3, bass textures place the root in E1–D#2, and drum patterns use a sixteenth-note grid with one hit at a time. Every genotype is thus a valid arrangement, and a validator checks the chip rules before export: one note at a time per channel, pulse pitches in MIDI 33–108, triangle pitches in

Gene	Channel	Values
P2	pulse 2	hold, arp16, arp32, echo, third
TR	triangle	root, octave pump, root–fifth, walking
NO	noise	none, half time, 8th beat, 16th beat, fill
OCT	pulse 1	as written, +12, −12 semitones

Table 1. The four genes of each bar. P1 always plays the melody; the GA chooses its octave.

21–108 [1], chord tones on beats 1 and 3, and complete bars.

3.2 Fitness

The fitness is the weighted mean of seven criteria scored in $[0, 1]$; a criterion that does not apply to a tune (contrast without a chorus) is left out. With $a_c(b)$ the activity of channel c in bar b (onsets per bar divided by 16, capped at 1): *complementarity* rewards $a_{P2}(b) \approx 1 - a_{P1}(b)$, so the harmony fills the gaps of the melody; *register* is the share of sixteenth steps where P2 sounds below P1, a voice-crossing penalty as in GA harmonization [7]; *coherence* is the share of adjacent bars inside a phrase with the same textures (a fill matches any drum pattern), echoing GenJam's phrase structure [8]; *contrast* is $0.5 + \bar{d}_{chorus} - \bar{d}_{verse}$ clipped to $[0, 1]$, with d the mean activity of P2, TR and NO; *bass smoothness* scores the leap between the last bass note of a bar and the first of the next (1 up to a fourth, 0 from an octave); *idiom* rewards arpeggios over bars with a single chord and fills exactly at phrase ends [2]; *fidelity* is the share of bars where the melody stays in its written octave, which prevents the degenerate plan that raises the whole melody to win the register criterion. The default weights are 1, with 0.5 for fidelity. Since every realization depends only on its bar and its gene, the system precomputes per-bar score tables and evaluates a whole population with array indexing.

3.3 Genetic Operators

The initial population has 60 random genotypes. Parents are drawn by roulette selection with windowing (probability proportional to the fitness minus the lowest fitness of the generation), which keeps the selection pressure when the weighted mean gives close values; the two best genotypes pass unchanged (elitism). One-point crossover cuts only between bars [6], and mutation replaces each gene by another valid value with probability 0.01, close to $1/L$ for $L = 4\times$ bars. The GA stops after 1000 generations or after 100 generations without improvement. All randomness comes from one seeded generator, and each run stores its seed, hyperparameters, input hash and code commit.

3.4 Inputs and Rendering

ABC lead sheets give the melody, the chords and the part labels directly. A multi-track MIDI song is first reduced to a lead sheet. The chosen melody track or tracks are merged by a skyline (the highest note wins) and snapped to a sixteenth-note grid. One chord per half bar is chosen among major, minor, dominant-seventh and minor-seventh chords: the other tracks vote for pitch classes in proportion to how long they sound, bass tracks count twice, and a chord whose root is the bass note gets a bonus. The arrangement is rendered twice from the same MIDI file. One rendering uses FluidSynth with General MIDI timbres. The other uses our own chip synthesizer, with pulse waves of 25% and 12.5% duty, the 4-bit 32-step triangle and 15-bit LFSR noise [1]. A video draws each channel as a glowing step line in sync with the audio.

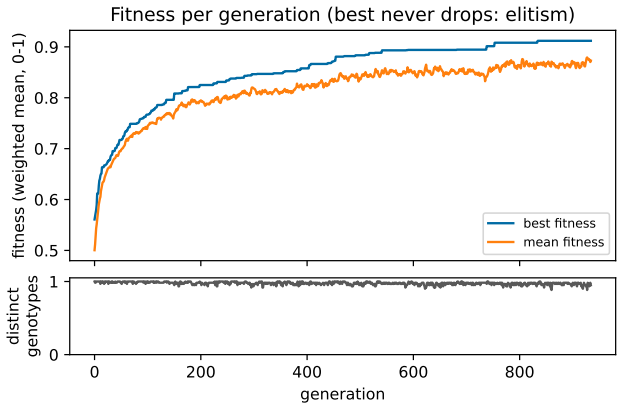


Figure 1. Korobeiniki, seed 1. Top: best and mean fitness per generation. Elitism keeps the best curve monotonic. Bottom: share of distinct genotypes in the population.

4. RESULTS

Setup. We arranged two traditional tunes with well-known 8-bit versions: *Ievan Polkka*, with the melody as sung by Loituma, in E-flat minor at 110 BPM, with form A B A′ played twice (24 bars), and *Korobeiniki*, the Tetris theme, in A minor at 150 BPM, with form A A B A (32 bars). The B parts are labelled chorus. Every run uses the default configuration unless stated otherwise, and takes about 8 s on a laptop, including audio rendering with FluidSynth and a General MIDI soundfont (square lead for the pulses, synth bass for the triangle, a drum kit for the noise). All runs stopped by stagnation, after 330 to 934 generations.

Evolution. Figure 1 shows a typical run. The best fitness rises from 0.56 in the random generation 0 to 0.91, and the mean fitness follows it closely. The share of distinct genotypes stays above 0.83: the plans become similar, but mutation keeps most of them different. Between generation 0 and the final plan, the coherence of Korobeiniki goes from 0 to 0.96 and its fidelity from 0.38 to 1.

Is the GA needed? Table 2 compares five seeds with a random search that scores as many random plans as the first GA run did (33,060 and 56,100 plans). Random search stays near 0.6 because it never finds coherent phrases (coherence 0), while every GA run reaches 0.85 to 0.94. The seeds end on different plans: on average, 29% (Ievan Polkka) and 38% (Korobeiniki) of the genes differ between two seeds. The same configuration therefore yields different arrangements, as the assignment requires.

Ablation and hyperparameter variation. Setting one

	Ievan Polkka	Korobeiniki
GA, 5 seeds (mean, min–max)	0.92 (0.90–0.94)	0.88 (0.85–0.91)
Random search, same evaluations	0.62	0.60
Genes that differ between seeds	29%	38%
Ablation, seed 1: score of the removed criterion (default → weight 0)		
coherence	0.94 → 0.44	0.96 → 0.25
contrast	1.00 → 0.52	0.75 → 0.56
bass smoothness	0.99 → 0.60	1.00 → 0.77
idiom	0.98 → 0.56	0.97 → 0.67
fidelity	1.00 → 0.25	1.00 → 0.22
register	1.00 → 0.87	1.00 → 0.99
complementarity	0.63 → 0.62	0.75 → 0.73

Table 2. Fitness with the default weights, and a weight ablation. Removing a criterion lowers its own score, except for complementarity and register, which the other criteria already satisfy.

weight to 0 lowers that criterion (Table 2). Without fidelity, the GA moves most bars of the melody by an octave. Without coherence, it changes textures inside phrases. Without contrast, the chorus loses its denser accompaniment. Two criteria barely move. Complementarity is bounded by the busy melodies, and register holds because P2 sits below the melody by construction. The submitted pieces vary the seed (Korobeiniki, seeds 1 and 2) and the contrast weight (Ievan Polkka, weights 1 and 0). With contrast at 0, the GA prefers a parallel line of chord tones under the melody instead of a held verse and an arpeggiated chorus.

Listening. In Ievan Polkka (seed 1), the first A part holds the chord root over a walking bass and a half-time beat, the B parts switch to fast arpeggios with a sixteenth-note beat, and a snare fill closes every phrase. Korobeiniki keeps arpeggios throughout and saves the dense drums for the slow bridge, where the melody holds long notes. The noise channel sounds 39% to 46% of the time, close to the 39% measured in NES game music, but our pulse and triangle channels never rest (100%, against 84% and 70% in that corpus [1]).

A MIDI song. We also arranged bars 8–56 of *Voa Voa* (Chiclete com Banana): a verse, a pre-chorus and a chorus that modulates. The source is a fan-made MIDI file recorded live. It states 120 BPM, but its drum hits fit a sixteenth-note grid at 110.1 BPM (strength 0.90 against 0.04 for the stated tempo), so the reader re-estimates the grid before cutting bars. The arrangement reaches a lower fitness (0.80, after 1784 generations). The sax melody sits around octave 3, so keeping it in its written register conflicts with keeping P2 below it. The GA trades fidelity (0.73) against register (0.93), and the longer form lowers coherence (0.58).

5. DISCUSSION AND FUTURE WORK

The GA turns a lead sheet into a plausible 8-bit arrangement in seconds, and the experiments show that the search, not chance, produces the result. The approach has clear limits. The fitness is a weighted sum of hand-written heuristics, a common choice that hides the conflicts between objectives [13]. A Pareto front [5, 6] would expose these trade-offs. Automatic chord detection on MIDI songs is approximate (a few major chords come out minor). The textures come from a fixed vocabulary, so the GA chooses between arranging gestures but does not invent new ones. The channels never rest. Only 4/4 lead sheets are supported. General MIDI timbres only approximate the chip, which also has duty cycles and a sample channel that we do not model. Future work includes a fitness learned from the NES Music Database [1], evolving rhythmic and melodic variations of the P2 and TR lines, and a listening test with A/B comparisons, as in [11].

6. AI USAGE STATEMENT

This work was developed with Claude Code (Anthropic) under the author's direction. The tool was used for brainstorming, for checking the design against the course material and the literature, for writing the specification, the code and its tests, for transcribing the two lead sheets and for drafting this text. The author chose the method, the constraint and the songs, listened to every input and output, and reviewed the results.

7. REFERENCES

[1] C. Donahue, H. H. Mao, and J. McAuley, "The NES music database: A multi-instrumental dataset with expressive performance attributes," in *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, Paris, France, 2018. [Online]. Available: https://arxiv.org/abs/1806.04278

[2] K. Driscoll and J. Diaz, "Endless loop: A brief history of chiptunes," *Transformative Works and Cultures*, vol. 2, 2009.

[3] K. Collins, "In the loop: Creativity and constraint in 8-bit video game audio," *Twentieth-Century Music*, vol. 4, no. 2, pp. 209–227, 2007.

[4] D. R. Tuohy and W. D. Potter, "GA-based music arranging for guitar," in *2006 IEEE International Conference on Evolutionary Computation (CEC)*. IEEE, 2006, pp. 1065–1070.

[5] G. Carpentier, G. Assayag, and E. Saint-James, "Solving the musical orchestration problem using multiobjective constrained optimization with a genetic local search approach," *Journal of Heuristics*, vol. 16, no. 5, pp. 681–714, 2010.

[6] A. R. R. Freitas and F. G. Guimarães, "Melody harmonization in evolutionary music using multiobjective genetic algorithms," in *Proceedings of the 8th Sound and Music Computing Conference (SMC)*. Padova, Italy: Zenodo, 2011.

[7] S. Phon-Amnuaisuk, A. Tuson, and G. Wiggins, "Evolving musical harmonisation," in *Artificial Neural Nets and Genetic Algorithms*. Springer Vienna, 1999, pp. 229–234.

[8] J. A. Biles, “GenJam: A genetic algorithm for generating jazz solos,” in *Proceedings of the International Computer Music Conference (ICMC)*. Aarhus, Denmark: International Computer Music Association, 1994, pp. 131–137. [Online]. Available: https://genjam.org/wp-content/uploads/2019/07/bilesicmc94.pdf

[9] E. Özcan and T. Erçal, “A genetic algorithm for generating improvised music,” in *Artificial Evolution*, ser. Lecture Notes in Computer Science, vol. 4926. Springer, 2008, pp. 266–277.

[10] M. A. Kaliakatsos-Papakostas, M. G. Epitropakis, A. Floros, and M. N. Vrahatis, “Interactive evolution of 8-bit melodies with genetic programming towards finding aesthetic measures for sound,” in *Evolutionary and Biologically Inspired Music, Sound, Art and Design (EvoMUSART)*, ser. Lecture Notes in Computer Science, vol. 7247. Springer, 2012, pp. 141–152.

[11] C. Donahue, H. H. Mao, Y. E. Li, G. W. Cottrell, and J. McAuley, “LakhNES: Improving multi-instrumental music generation with cross-domain pre-training,” in *Proceedings of the 20th International Society for Music Information Retrieval Conference (ISMIR)*, Delft, The Netherlands, 2019. [Online]. Available: https://arxiv.org/abs/1907.04868

[12] L. N. Ferreira and J. Whitehead, “Learning to generate music with sentiment,” in *Proceedings of the 20th International Society for Music Information Retrieval Conference (ISMIR)*, Delft, The Netherlands, 2019, pp. 384–390.

[13] J. D. Fernández and F. Vico, “AI methods in algorithmic composition: a comprehensive survey,” *Journal of Artificial Intelligence Research*, vol. 48, pp. 513–582, 2013.